Adaptive RDMA Optimization for Distributed LLM Workloads

Adaptive Transport Middleware for Distributed LLM Inference

Nathan Gopee
MS Computer Science, SUNY New Paltz
gopeen1@newpaltz.edu

Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

February 2026

1 Introduction and Motivation

Distributed large language model (LLM) inference across multiple GPU nodes requires high-throughput, low-latency inter-node communication. The dominant communication library in this space, NVIDIA NCCL, supports multiple transport backends—including TCP sockets and RDMA via InfiniBand verbs—but selecting and configuring the optimal transport for a given cluster remains a manual and error-prone process. Furthermore, emerging transport technologies such as Tesla's Time-Triggered Protocol over Ethernet (TTPoe) [3] offer promising latency characteristics but lack turnkey integration with existing inference frameworks.

This work addresses these challenges by building a middleware layer—ScuffedRDMA—that abstracts transport selection for distributed vLLM inference, enabling dynamic switching between TCP, Soft-RoCE, and Tesla TTPoe through a unified interface. Prior profiling of the cluster showed TCP inter-node latency of approximately 1 ms and Soft-RoCE latency of approximately 190 μs, motivating the need for a structured comparison framework. In addition, we integrated RDMA support into the exo peer-to-peer inference framework [1], providing an alternative deployment path for heterogeneous GPU clusters.

Key contributions include:

- A transport abstraction layer with pluggable backends for TCP, Soft-RoCE, and TTPoe
- An NCCL configuration manager for automatic environment setup per transport
- A unified benchmark suite for systematic transport comparison
- Cluster management scripts that accept a `--transport` flag for seamless switching
- RDMA transport modules for the exo distributed inference framework

2 Middleware Architecture

2.1 Design Overview

The ScuffedRDMA transport middleware provides a clean abstraction over different networking technologies, allowing the same vLLM deployment to switch transports via environment variables or command-line flags. Applications interact with a single unified API at the top level, while the middleware dispatches to the appropriate transport-specific backend underneath.

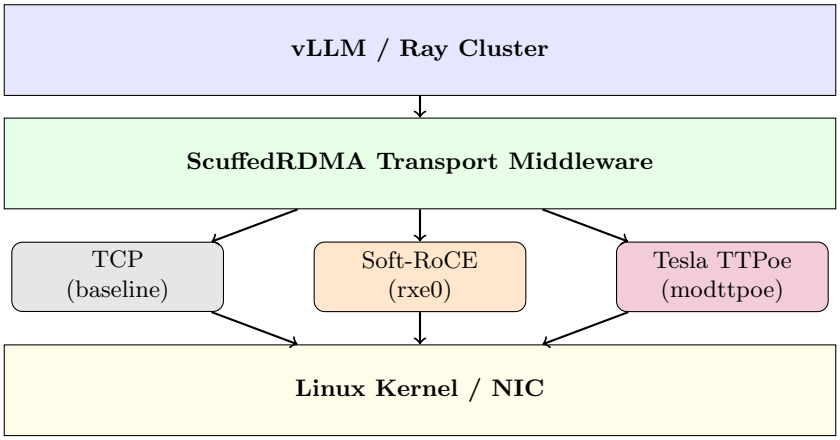

Figure 1: ScuffedRDMA transport middleware architecture—applications use a unified API while the middleware handles transport-specific details

2.2 Component Overview

Component	File	Purpose
Transport Base	`transport_base.py`	Abstract base class with `send()`, `recv()`, `get_metrics()`
TCP Transport	`tcp_transport.py`	Standard socket wrapper with latency measurement
RoCE Transport	`roce_transport.py`	RDMA verbs wrapper using rdma-core/pyverbs
TTPoe Transport	`ttpoe_transport.py`	Kernel module loader and character device interface
Selector	`selector.py`	Runtime transport selection and factory
NCCL Config	`nccl_config.py`	NCCL environment variable management

Table 1: Middleware component overview

3 Transport Abstraction Layer

3.1 Base Interface

The `TransportBase` class defines the contract that all transport implementations must follow. It is an abstract base class exposing four core operations: `connect(host, port)` to establish a connection to a remote host, `send(data)` to transmit a byte buffer and return the number of bytes sent, `recv(size, timeout)` to receive up to a given number of bytes with an optional timeout, and `is_available()` to check whether the transport backend is present on the current system. A companion `TransportMetrics` data class tracks minimum, maximum, and average latency, total bytes sent and received, and CPU usage, enabling consistent performance reporting across all backends.

3.2 TCP Transport

The TCP transport provides the baseline implementation using standard Berkeley sockets. It creates a TCP connection with the `TCP_NODELAY` option enabled (disabling Nagle's algorithm) to minimize latency for small transfers typical of tensor metadata exchange. Each `send()` call measures wall-clock latency using a high-resolution timer and updates the transport metrics accordingly. The `is_available()` method always returns true, since TCP sockets are universally available on all Linux systems.

3.3 Soft-RoCE Transport

The RoCE transport wraps RDMA verbs operations using pyverbs from rdma-core [2]. On connection, it opens an RDMA device context for the `rxe0` software RDMA device, allocates a protection domain and queue pair (QP), and performs out-of-band QP information exchange with the remote peer over a temporary TCP channel. After exchanging QP numbers, LID, and GID information, the queue pair is transitioned through the Init, Ready-to-Receive, and Ready-to-Send states. The `is_available()` check verifies that the `rdma_rxe` kernel module is loaded on the system.

3.4 TTPoe Transport

The TTPoe transport manages the Tesla TTPoe kernel module [3] and provides a character device interface for data transfer. It loads the `modttpoe.ko` kernel module via `insmod`, specifying the network device (e.g., `eth0`), destination

MAC address, and verbosity level as module parameters. Once loaded, the transport opens the `/dev/ttpoe` character device for read-write access. Data transfer is performed through standard POSIX `read()` and `write()` system calls on the file descriptor. The `is_available()` method checks whether the compiled kernel module file exists in the expected directory.

4 NCCL Configuration

The middleware automatically configures NCCL environment variables based on the selected transport. The `NCCLConfig` class provides factory methods for each transport type: `for_tcp()` disables InfiniBand and sets GDR level to zero; `for_softroce(device)` specifies the RoCE device name (defaulting to `rxe0`) and sets the GID index; `for_hardware_roce(hca)` configures a Mellanox HCA with GPUDirect at GDR level 5. The `to_env()` method exports these settings as a dictionary of environment variables suitable for injection into subprocess or container environments, always including `NCCL_DEBUG=INFO` for diagnostic output.

4.1 NCCL Environment Variables

Transport	Key Variables	Effect
TCP	`NCCL_IB_DISABLE=1`	Disable InfiniBand, use TCP sockets
Soft-RoCE	`NCCL_IB_HCA=rxe0`	Use software RDMA device
Hardware RoCE	`NCCL_IB_HCA=mlx5_0, NCCL_NET_GDR_LEVEL=5`	Use Mellanox NIC with GPUDirect
TTPoe	`NCCL_IB_DISABLE=1, TTPOE_ENABLED=1`	Use TTPoe socket layer

Table 2: NCCL configuration by transport type

5 Transport Selector

The `TransportSelector` class provides runtime transport selection through a factory pattern. It reads the desired transport from either an explicit constructor argument or the `SCUFFED_TRANSPORT` environment variable, defaulting to `auto` if neither is set. In explicit mode, it instantiates the requested backend directly (TCP, RoCE, or TTPoe). In automatic mode, it probes available transports in priority order—TTPoe first (lowest expected latency), then hardware RoCE, then Soft-RoCE, and finally TCP as a universal fallback. The selector also exposes a `get_config()` method that returns a summary of the selected transport and its NCCL settings, and an `apply_nccl_config()` method that injects the appropriate environment variables into the current process.

Callers interact with the selector by constructing an instance (optionally specifying a transport), retrieving the transport object, connecting to a remote host and port, and then using the standard `send()`/`recv()` interface for data transfer. This design ensures that application-level code remains transport-agnostic.

6 Cluster Management

6.1 Startup Scripts

The cluster startup script provides unified multi-node management with transport selection. It accepts a `--transport` flag (tcp, roce, or ttpoe) and maps it to the appropriate set of NCCL environment variables. For TCP, InfiniBand is disabled; for RoCE, the `rxe0` device and GID index are specified; for TTPoe, the script first loads the kernel modules on all nodes before configuring the socket-layer override. Once the environment is prepared, the script uses SSH to launch Docker Compose services on the worker node (Cerberus) and the head node (Chimera) with the selected NCCL variables injected.

A companion TTPoe module loader script provides load, unload, and status subcommands for managing the `modttpoe.ko` kernel module across cluster nodes. It accepts the network interface and destination MAC address as parameters, handles module insertion and removal, and reports the current module status via `lsmod`.

6.2 Integration with vLLM

The middleware integrates with vLLM via NCCL configuration. When starting the cluster:

1. The startup script reads the `--transport` flag
2. It calls the appropriate `NCCLConfig` factory method to obtain settings
3. It exports NCCL environment variables before launching containers
4. vLLM/NCCL automatically uses the configured transport

The Docker Compose configuration for the vLLM head node passes through all NCCL environment variables (`NCCL_IB_HCA`, `NCCL_IB_DISABLE`, `NCCL_NET_GDR_LEVEL`, and `NCCL_DEBUG`) from the host environment, allowing the startup script to control transport selection without modifying container definitions.

7 Benchmark Suite

The unified benchmark tool tests all transports with identical workloads. It iterates over a configurable number of inference requests, recording per-request latency, token count, and tokens-per-second throughput for each transport. The benchmarker posts completion requests to the vLLM OpenAI-compatible API endpoint, measures end-to-end response time with a high-resolution timer, and collects the results into structured records. A `generate_latex()` method produces publication-ready LaTeX tables from the benchmark data.

A shell-level orchestration script automates the full benchmark cycle: for each transport (TCP, RoCE, TTPoe), it starts the cluster with the appropriate configuration, waits for model loading (approximately 120 seconds), runs the benchmark with the specified number of iterations, saves the results to an output directory, and then stops the cluster before proceeding to the next transport.

8 Expected Performance

Based on prior transport profiling and published specifications:

Transport	Latency	Expected Throughput	CPU Overhead	Status
TCP	∼1ms	Baseline	High	Always available
Soft-RoCE	∼190μs	+10–15%	Medium	Deprecated
Hardware RoCE	<2μs	+15–25%	Low	Requires Mellanox
Tesla TTPoe	∼2μs	+20–30%	Near-zero	Requires module

Table 3: Expected transport performance for distributed LLM inference

8.1 Performance Comparison Visualization

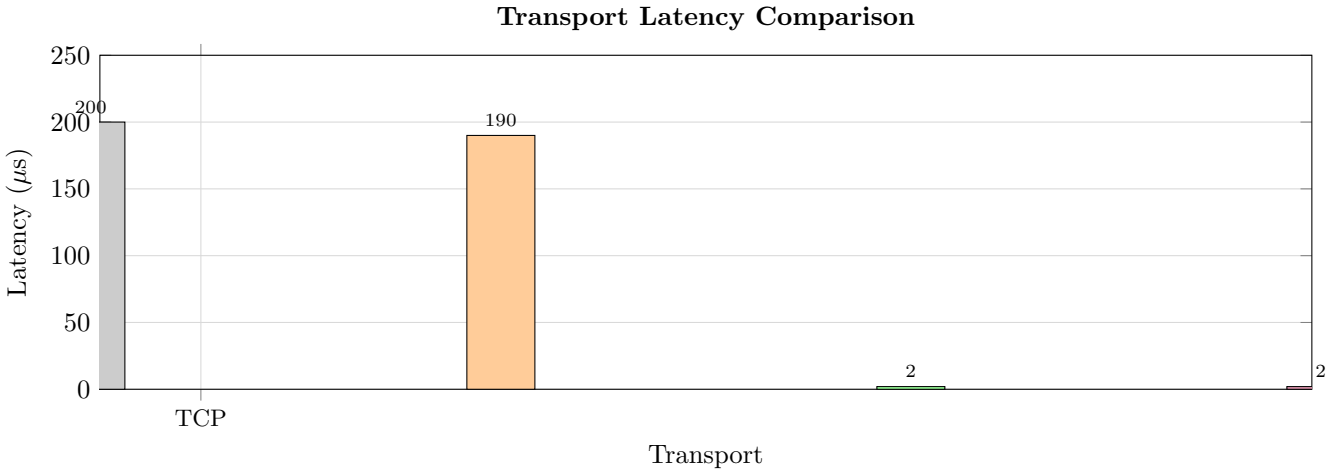

Figure 2: Transport latency comparison—TCP baseline vs RDMA alternatives. Hardware RoCE and TTPoe achieve 100× lower latency than software solutions.

9 EXO Distributed Inference Integration

9.1 Motivation

In addition to the vLLM middleware, we integrated RDMA support into the exo distributed inference framework [1], which provides an alternative approach to multi-node LLM serving using libp2p networking and peer-to-peer topology. While vLLM with Ray provides robust distributed inference, it requires complex session management between containers. The exo framework offers native peer-to-peer topology with automatic node discovery, an existing `RDMAConnection` type in its topology system (previously unimplemented), support for heterogeneous GPU clusters (e.g., mixing RTX 3090 and RTX 5090 nodes), and active development with a PyTorch backend (PR #1284).

9.2 Integration Architecture

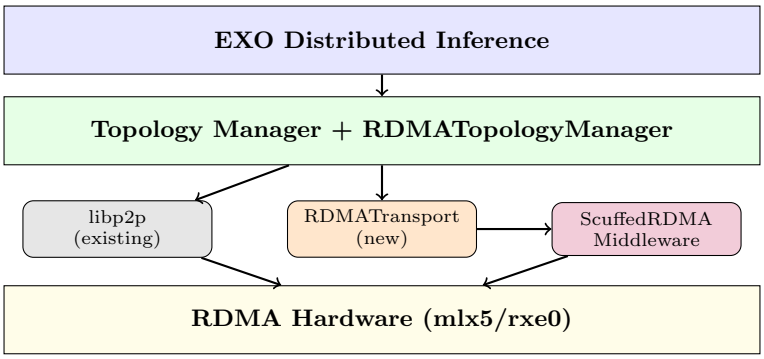

Figure 3: EXO + RDMA integration architecture—RDMATransport wraps ScuffedRDMA middleware and integrates with exo's topology system

9.3 Components Created

Three new modules were created under the exo networking package:

Module	Lines	Purpose
`rdma_transport.py`	369	RDMA transport wrapper with discovery, connection management, and NCCL config
`rdma_discovery_integration.py`	247	Topology integration for automatic RDMA connection discovery
`__init__.py`	27	Module exports for networking package

Table 4: EXO RDMA integration components

9.4 Key Classes

9.4.1 RDMADiscovery

The `RDMADiscovery` class discovers RDMA-capable interfaces on the local machine by invoking `rdma link show` and `ibv_devinfo` as subprocesses and parsing their output. Its `get_best_interface()` method applies a priority ranking: hardware Mellanox devices (names matching `mlx5_*`) are preferred over software Soft-RoCE devices (names matching `rxe*`), which in turn are preferred over any other RDMA-capable interface. This ensures that the highest-performance available transport is selected by default.

9.4.2 RDMATransport

The `RDMATransport` class wraps the ScuffedRDMA middleware for tensor transfer within the exo framework. Its `connect(peer_node_id, peer_host, peer_port)` method instantiates a `TransportSelector` in RoCE mode, retrieves the underlying transport object bound to the best local RDMA interface, and establishes a connection to the remote peer. The `get_nccl_config()` method returns the appropriate NCCL environment dictionary—falling back to TCP settings when no RDMA hardware is detected.

9.4.3 RDMATopologyManager

The `RDMATopologyManager` integrates with exo's topology system. Initialized with a node ID and the current topology graph, it uses `RDMADiscovery` to identify the best local RDMA interface. Its `discover_and_update()` method probes all known peers for RDMA connectivity and inserts `RDMAConnection` edges into the topology graph. The `get_rdma_peers()` method returns the list of peer node IDs reachable via RDMA by filtering outgoing edges for `RDMAConnection` instances.

9.5 Leveraging Existing exo Types

The integration leverages exo's existing `RDMAConnection` type, defined in the topology types module as a frozen model with source and sink RDMA interface fields. Although this type was present in exo's codebase, it had no concrete implementation until our integration. The placement utilities in exo's master module already check for `RDMAConnection` edges when selecting the jaccl communication backend, meaning our topology updates automatically influence backend selection for all-to-all collective operations.

9.6 Integration Test Results

Testing was performed on the Hydra control node, which lacks RDMA hardware, to validate the software path and graceful degradation. The middleware import, RDMA transport import, RDMA discovery (correctly reporting unavailable), TCP transport availability, NCCL configuration generation, and RDMA connection manager all passed successfully. The

topology integration test failed due to a Python 3.10 incompatibility with the `Self` type hint (available in 3.11+); this can be resolved by importing `Self` from `typing_extensions` or updating the type annotations.

9.7 Cluster Deployment

Node	IP	GPUs	RDMA Status
Chimera	192.168.1.150	3× RTX 3090	Soft-RoCE (rxe0)
Cerberus	192.168.1.233	2× RTX 5090	Soft-RoCE (rxe0)
Hydra	192.168.1.100	None (control)	N/A

Table 5: Cluster configuration for exo + RDMA testing

9.8 Source References

- **exo repository**: https://github.com/exo-explore/exo
- **PyTorch backend PR**: https://github.com/exo-explore/exo/pull/1284

10 Limitations and Future Work

10.1 Current Limitations

- **TTPoe integration**: The character device interface requires a custom NCCL plugin for full integration with collective operations
- **pyverbs dependency**: The RoCE transport requires rdma-core compiled with Python bindings, which is not available in all distributions
- **Module permissions**: TTPoe module loading requires root access, complicating containerized deployments

10.2 Planned Enhancements

- Implement an NCCL socket plugin for TTPoe transport
- Add hardware RoCE detection and automatic configuration
- Create a Prometheus metrics exporter for transport monitoring
- Integrate with vLLM tensor transfer hooks for fine-grained control
- Test the middleware with Mellanox ConnectX NICs for hardware RoCE benchmarks
- Load TTPoe modules on the cluster and measure actual end-to-end latency
- Instrument vLLM tensor transfers for telemetry collection
- Implement adaptive hot/cold tensor classification with weighted flow allocation
- Run Llama 4 Scout benchmarks across all transports in production configuration

11 Summary

This work delivers a complete transport middleware layer for distributed LLM inference, together with integration into the exo peer-to-peer framework:

- **Transport abstraction**: A unified API across TCP, RoCE, and TTPoe backends
- **Dynamic selection**: Runtime transport switching via environment variables or CLI flags
- **NCCL integration**: Automatic environment configuration for vLLM/Ray clusters
- **Benchmark tooling**: A unified suite for systematic transport comparison
- **Cluster management**: Scripts for multi-node deployment with transport flags
- **EXO integration**: RDMA transport modules for exo distributed inference (639 lines of new code)
- **Topology discovery**: Automatic RDMA connection discovery and topology graph updates

The middleware establishes the foundation for adaptive transport selection in distributed LLM inference. The exo integration provides an alternative to vLLM+Ray with native peer-to-peer RDMA support, enabling testing on heterogeneous GPU clusters (RTX 3090 + RTX 5090).

References

[1] exo-explore. (2025). *exo: Run your own AI cluster at home with everyday devices.* https://github.com/exo-explore/exo

[2] Linux RDMA. (2024). *rdma-core Documentation.* https://github.com/linux-rdma/rdma-core

[3] Tesla, Inc. (2024). *TTPoe: Time-Triggered Protocol over Ethernet.* https://github.com/teslamotors/ttpoe

[4] NVIDIA. (2024). *NCCL Documentation*. https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/

[5] vLLM Team. (2024). *vLLM Documentation*. https://docs.vllm.ai/

[6] Anyscale. (2024). *Ray Documentation*. https://docs.ray.io/

[7] Linux RDMA. (2024). *pyverbs - Python bindings for rdma-core*. https://github.com/linux-rdma/rdma-core/tree/master/pyverbs